

МИНИСТЕРСТВО ЛЕСНОГО ХОЗЯЙСТВА И ОХРАНЫ ОБЪЕКТОВ
ЖИВОТНОГО МИРА НИЖЕГОРОДСКОЙ ОБЛАСТИ

Государственное бюджетное профессиональное образовательное учреждение
Нижегородской области «КРАСНОБАКОВСКИЙ ЛЕСНОЙ КОЛЛЕДЖ»
(ГБПОУ НО «КБЛК»)

ДЕНЬ 9

**Анализ технического задания на программный продукт. Изучение
требований к взаимодействию компонент**

Цель: проанализировать техническое задание на систему онлайн-бронирования столиков в ресторане, выделить все компоненты системы, определить требования к их взаимодействию и визуализировать полученные результаты в виде таблицы и диаграммы последовательности.

Выполнила: Аракчеева Екатерина,

Студентка 24-ИС.

Руководитель: Александр Вениаминович Торопов

р.п. Красные Баки

2026 г.

Выделенные компоненты

В результате анализа технического задания выделены следующие компоненты системы:

1. **Клиентское приложение** - веб-сайт или мобильное приложение, через которое клиент выбирает дату, время, количество гостей и создаёт бронь.
2. **Сервер бронирования** - центральный сервер, обрабатывающий запросы от клиентов и менеджеров, координирующий работу всех модулей.
3. **Модуль управления залом** - компонент, проверяющий доступность столиков на выбранное время и резервирующий их.
4. **База данных бронирований** - хранилище информации о всех бронированиях (клиент, дата, время, статус).
5. **Модуль уведомлений** - компонент, отправляющий клиенту SMS и email-подтверждение о бронировании.
6. **Веб-интерфейс менеджера** - отдельная панель для менеджера ресторана для просмотра, редактирования и отмены броней в реальном времени.

Таблица 1 - Требования к взаимодействию компонентов

№	Компонент-источник	Компонент-приёмник	Тип взаимодействия	Протокол / API	Формат данных	Доп. условия (таймаут, безопасность)
1	Клиентское приложение	Сервер бронирования	Синхронный	REST API (HTTPS)	JSON	checkAvailabilit, createReservation
2	Сервер бронирования	Модуль управления залом	Синхронный	gRPC	Protocol Buffers	Таймаут 2 сек.
3	Сервер бронирования	База данных бронирований	Синхронный	SQL	Таблицы	Сохранение брони
4	Сервер бронирования	Модуль уведомлений	Асинхронное	RabbitMQ	JSON	Очередь notification_queue
5	Веб-интерфейс менеджера	Сервер бронирования	Асинхронное	WebSocket	JSON	Обновления в реальном времени

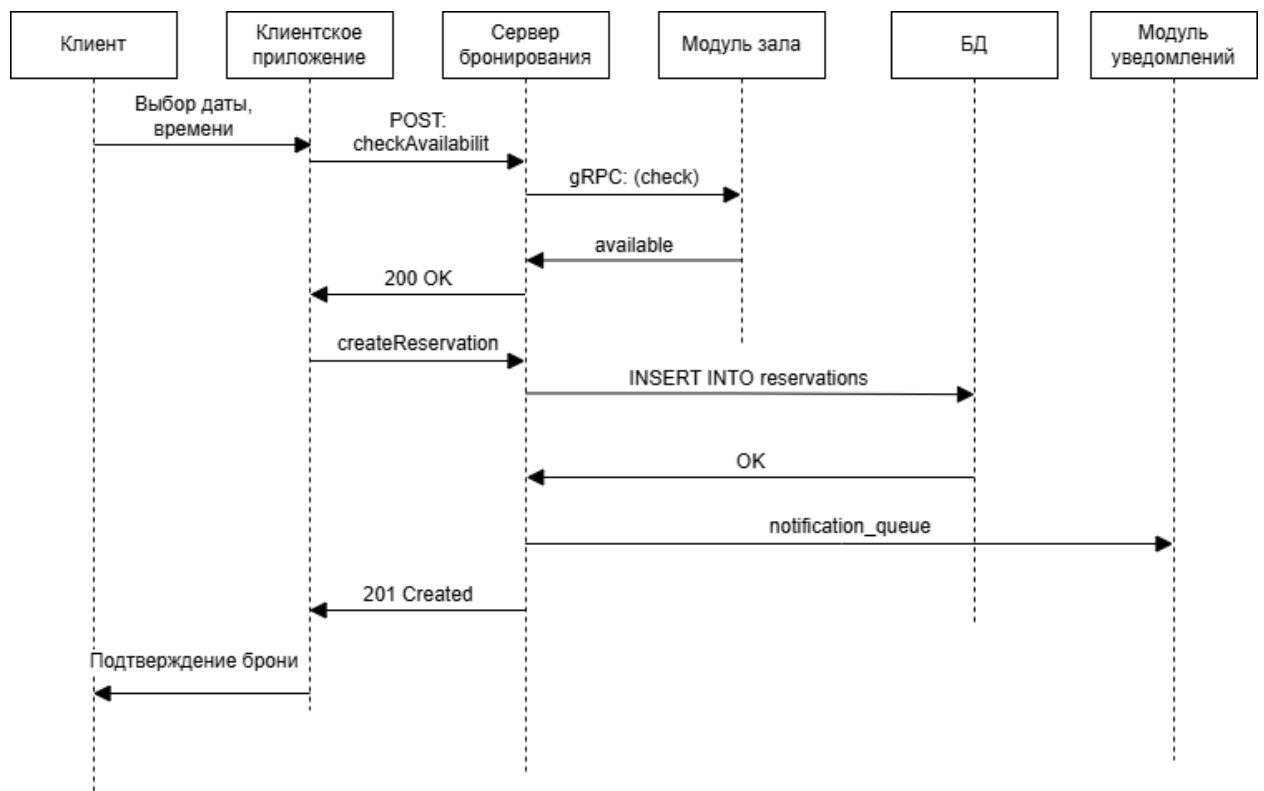


Рисунок 1 - Диаграмма последовательности

Ответы на вопросы:

1. Обязательные разделы ТЗ по ГОСТ 19.201-78 (8 разделов):

Введение, основания, назначение, требования к программе, требования к документации, тех.-эконом. показатели, стадии разработки, порядок приёмки. Требования к взаимодействию - в разделе 4 «Требования к программе».

2. Функциональные vs нефункциональные:

Функциональные - ЧТО делает (например, «проверка доступности столика»).
Нефункциональные - КАК делает (например, «таймаут ответа 2 секунды» - это нефункциональное требование к взаимодействию).

3. Виды взаимодействия:

- **Синхронный** - ждёт ответа (REST, gRPC, вызов БД).
Используется, когда нужен результат сразу.
- **Асинхронный** - отправил и забыл (очереди сообщений, уведомления). Используется для долгих операций.

4. API - набор правил и операций для взаимодействия компонентов. В ТЗ описывается таблицей: операция, метод, входные/выходные данные, формат.

5. Почему важно указывать протокол и формат: чтобы разработчики реализовали взаимодействие одинаково, и компоненты смогли понять друг друга (иначе JSON не сойдётся с XML, HTTP с WebSocket).

6. Проблемы при нечётких требованиях: Несовместимые форматы, неизвестный протокол, непонятная обработка ошибок, проблемы с отладкой, задержки, отсутствие аутентификации. Интеграция либо не заработает, либо будет работать с ошибками.

Вывод: в ходе анализа технического задания на систему онлайн-бронирования столиков было выделено 6 компонентов и определены требования к их взаимодействию. Наиболее критичными требованиями являются синхронный вызов модуля зала с таймаутом 2 секунды (при задержке бронь может быть ошибочно отклонена), асинхронное взаимодействие с модулем уведомлений через RabbitMQ (требует контроля очереди) и WebSocket-соединение для менеджера (необходимо для обновлений в реальном времени). Среди потенциальных проблем выявлены отсутствие описания обработки ошибок, неспецифицированный формат JSON для API, отсутствие механизма повторных попыток при сбоях, недостаток требований к производительности и неуточнённый формат даты и времени. В целом техническое задание содержит достаточную информацию для начала разработки, однако требует доработки в части уточнения форматов данных, обработки ошибок и политики повторных попыток.